PUBLIC INTELLIGENCE

SaaS Platform — Full Technical Blueprint

Open-Source OSINT Aggregation System

Version 1.0 | July 2026

✓ **100% Free & Open-Source Stack**

Intended Audience: Software Engineers & AI Agents

Table of Contents

1. Executive Summary

Public Intelligence is a SaaS web application that enables a user to search for any person using one or more known identifiers (name, email, username, social media URL, company, city, etc.) and automatically aggregates all publicly available information about that individual from Google, LinkedIn, Facebook, Instagram, Twitter/X, GitHub, and other open internet sources.

The system presents consolidated results across multiple data categories — contact details, social profiles, professional history, and online presence — through a clean, searchable interface. The entire platform is built exclusively on free and open-source technologies with **zero paid API dependencies**.

> ***KEY PRINCIPLE:*** *This system only retrieves information that is publicly available. It does not bypass paywalls, authentication walls, or private data. Compliance with platform Terms of Service and applicable privacy law is mandatory.*

2. Product Goals & Core Features

2.1 Primary Goal

A user enters any known information about a target person (one field is mandatory). The system searches across multiple internet sources and returns a unified profile containing all discoverable public data.

2.2 Input Fields (Search Form)

The following fields are available on the search form. At least **ONE must be filled** before a search can be submitted:

Input Field	Description
Full Name	First and last name of the person
Email Address	Personal or work email (any format)
LinkedIn URL / Username	Full LinkedIn profile URL or handle
Facebook URL / Username	Full Facebook profile URL or handle
Instagram Username	@handle or full URL
Company Name	Current or past employer
Company Website	Official website domain (e.g. acme.com)
Personal Email	Separate field from work email
City	Current or known city of residence
Country	Country of residence or citizenship
GitHub Username	GitHub handle
Twitter/X Username	@handle

2.3 Output Data — Retrievable Checkboxes

The user selects which categories of information they wish to retrieve. Results are shown per category:

Output Category	What Is Returned
Personal Email	Discovered personal email addresses

Output Category	What Is Returned
Work Email	Professional/company email addresses
Phone Number	Any publicly listed phone numbers
LinkedIn Profile	Full LinkedIn profile data & activity
GitHub	Repositories, bio, contributions
Twitter/X	Tweets, bio, follower counts
Facebook	Public posts, bio, profile data
Instagram	Public posts, bio, follower counts
Personal Website	Any personal site or blog found
Company	Employer, role, company details

3. System Architecture

3.1 High-Level Architecture

The system follows a classic three-tier web application architecture with an asynchronous job queue for long-running scraping tasks.

Layer	Technology	Purpose
Presentation Layer	React.js (Next.js)	Browser-based SPA
API Layer	FastAPI (Python)	REST endpoints
Scraping Engine	Scrapy + Playwright	Async multi-source crawl
Job Queue	Celery + Redis	Background task management
Database	PostgreSQL + Redis	Results storage & caching
Search Index	Elasticsearch (optional)	Full-text search on results
Proxy Manager	Rotating proxies (free)	Avoid IP blocks
Deployment	Docker + Railway/Fly.io	Free-tier cloud hosting

3.2 Data Flow Diagram

```
User Browser
     |
     | HTTP POST /api/search  { fields, output_checkboxes }
     v
FastAPI Server
     |
     |-- validates input (at least 1 field required)
     |-- creates Job ID
     |-- pushes job to Celery queue (Redis broker)
     v
Celery Worker(s)  [runs in background]
     |
     |-- GoogleSearchScraper   --> discovers URLs, emails via dorks
     |-- LinkedInScraper       --> scrapes public profile page
     |-- FacebookScraper       --> scrapes public profile page
     |-- InstagramScraper      --> scrapes public profile page
     |-- TwitterScraper        --> scrapes public profile page
     |-- GitHubScraper         --> GitHub API (free, no auth needed)
     |-- HunterScraper         --> Hunter.io free plan (25 req/mo)
```

```
      |-- WhoIsScraper          --> WHOIS data for company domain
      |-- PhoneScraper          --> searches phone from indexed sites
      v
Result Aggregator
      |-- de-duplicates & merges results
      |-- scores confidence per data point
      |-- saves to PostgreSQL
      v
FastAPI Server  GET /api/results/{job_id}
      |
      v
User Browser  [displays results by category]
```

4. Full Technology Stack (100% Free)

4.1 Frontend

Package	Purpose	Source & License
Next.js 14	React framework with SSR/SSG	nextjs.org — MIT License
React 18	UI component library	reactjs.org — MIT License
Tailwind CSS	Utility-first CSS framework	tailwindcss.com — MIT License
shadcn/ui	Pre-built accessible UI components	ui.shadcn.com — MIT License
React Hook Form	Form state & validation	react-hook-form.com — MIT License
Zod	Schema-based input validation	zod.dev — MIT License
Axios	HTTP client for API calls	axios-http.com — MIT License
Socket.io Client	Real-time job progress updates	socket.io — MIT License
Lucide React	Icon library	lucide.dev — ISC License
Recharts	Charts for analytics (optional)	recharts.org — MIT License

4.2 Backend — API Server

Package	Purpose	Source & License
Python 3.12	Primary backend language	python.org — PSF License
FastAPI	High-performance async REST API	fastapi.tiangolo.com — MIT
Uvicorn	ASGI server for FastAPI	uvicorn.org — BSD
Pydantic v2	Data validation & serialization	docs.pydantic.dev — MIT
SQLAlchemy 2	ORM for database operations	sqlalchemy.org — MIT
Alembic	Database migrations	alembic.sqlalchemy.org — MIT
Celery 5	Distributed task queue	docs.celeryq.dev — BSD

Package	Purpose	Source & License
Redis-py	Redis client for Python	github.com/redis/redis-py — MIT
Python-dotenv	Environment variable management	pypi.org — BSD
Loguru	Structured logging	github.com/Delgan/loguru — MIT

4.3 Scraping Engine

Package	Purpose	Source & License
Scrapy 2.11	Async web scraping framework	scrapy.org — BSD
Playwright (Python)	Headless browser for JS-rendered pages	playwright.dev — Apache 2.0
scrapy-playwright	Scrapy + Playwright integration	github.com/scrapy-playwright — BSD
BeautifulSoup4	HTML parsing	pypi.org/bs4 — MIT
lxml	Fast XML/HTML parser	lxml.de — BSD
requests	Simple HTTP requests	python-requests.org — Apache 2.0
fake-useragent	Rotate user-agent strings	pypi.org — Apache 2.0
tldextract	Extract domain/subdomain from URLs	pypi.org — BSD
phonenumbers	Parse & validate phone numbers	github.com/daviddrysdale — Apache 2.0
python-whois	WHOIS lookups for domains	pypi.org — MIT

4.4 Data Storage

Technology	Purpose	Source & License
PostgreSQL 16	Primary relational database	postgresql.org — PostgreSQL License
Redis 7	Celery broker + result caching	redis.io — BSD
Elasticsearch 8 (optional)	Full-text search over results	elastic.co — SSPL / Elastic
SQLite (dev only)	Local dev database	sqlite.org — Public Domain

4.5 Free External APIs & Data Sources

API / Source	What It Provides	Cost
GitHub REST API v3	Public profile, repos, orgs — no auth needed	$0 — Free
Hunter.io Free Plan	Find work emails (25 req/month)	$0 — Free Tier
Google Custom Search API	10,000 free queries/day	$0 — Free Tier
DuckDuckGo Scrape	No API key needed — scrape results	$0 — No restriction
WHOIS XML API	500 free lookups/month	$0 — Free Tier
Clearbit Autocomplete	Company name enrichment	$0 — Free
Gravatar API	Email to avatar lookup	$0 — Free
CommonCrawl	Indexed web data — fully free	$0 — Open Data
Archive.org Wayback	Historical web snapshots	$0 — Free

4.6 DevOps & Deployment (All Free Tiers)

Tool	Purpose	Cost
Docker + Docker Compose	Containerization & local orchestration	$0 — Free
Railway.app	Free hosting for backend + DB + Redis	$0 — Free Tier
Vercel	Free frontend hosting for Next.js	$0 — Free Tier
GitHub Actions	CI/CD pipeline — 2000 min/month free	$0 — Free
GitHub	Source code repository	$0 — Free
Upstash Redis	Serverless Redis (10K cmd/day free)	$0 — Free
Neon.tech	Serverless PostgreSQL — free tier	$0 — Free
Cloudflare Pages	Alternative static hosting	$0 — Free

5. Scraper Design — Per Platform

5.1 Google / DuckDuckGo Scraper

The Google/DuckDuckGo scraper builds targeted search queries using Google Dorks to surface relevant public pages about the target. It is the primary discovery engine — all other scrapers receive URLs as leads from this step.

Google Dork Templates

```
# Dork construction examples (Python):
def build_dorks(name=None, email=None, company=None, city=None):
    dorks = []
    if name:
        dorks.append(f'"{name}" site:linkedin.com/in')
        dorks.append(f'"{name}" site:facebook.com')
        dorks.append(f'"{name}" site:twitter.com OR site:x.com')
        dorks.append(f'"{name}" site:github.com')
        dorks.append(f'"{name}" email OR contact OR phone')
        if company: dorks.append(f'"{name}" "{company}" email')
        if city:    dorks.append(f'"{name}" "{city}"')
    if email:
        dorks.append(f'"{email}"')
        dorks.append(f'"{email}" site:linkedin.com')
    return dorks
```

Scraping Method

Use DuckDuckGo HTML endpoint (`duckduckgo.com/html/`) — no API key required. Parse search result URLs and snippets with BeautifulSoup4. Rotate user-agents via `fake-useragent`. Add randomised delays (2–5 seconds) between requests to avoid blocks.

5.2 LinkedIn Scraper

LinkedIn requires headless browser rendering (Playwright) since it is JavaScript-heavy. The scraper accesses only public profiles — no login.

```
# scrapy-playwright spider pseudocode
class LinkedInSpider(scrapy.Spider):
    name = 'linkedin'
    custom_settings = {
        'DOWNLOAD_HANDLERS': {
            'https': 'scrapy_playwright.handler.ScrapyPlaywrightDownloadHandler'
        }
    }

    def start_requests(self):
        yield scrapy.Request(url=self.profile_url,
```

```
            meta={'playwright': True, 'playwright_include_page': True})

    async def parse(self, response):
        page = response.meta['page']
        await page.wait_for_selector('.top-card-layout__title', timeout=10000)
        name     = await page.inner_text('.top-card-layout__title')
        headline = await page.inner_text('.top-card-layout__headline')
        # ... further selectors for experience, education, skills
        await page.close()
```

Key data extracted: full name, headline, location, current and past employers, education history, skills list, profile URL, photo URL, connection count (if visible).

5.3 Facebook Scraper

Facebook public profiles are accessible without login. The scraper targets the `/about` section of public pages.

```
# Target URLs
https://www.facebook.com/{username}
https://www.facebook.com/{username}/about

# Key CSS selectors (update if Facebook changes markup):
# Name:     [data-testid="profile_name_in_profile_page"]
# Location: text near "Lives in", "From"
# Work:     text near "Works at"
# Bio:      [data-testid="profile_bio_field"]
```

5.4 Instagram Scraper

Instagram public profiles expose basic data in their HTML without authentication. Use requests + BeautifulSoup4 or Playwright for JS-rendered profiles.

```
# Target URL (JSON endpoint — may vary):
https://www.instagram.com/{username}/?__a=1&__d=dis

# Data available from public profile JSON:
# biography, external_url, full_name, is_verified,
# edge_followed_by (followers), edge_follow (following),
# profile_pic_url, username, recent posts (thumbnails only)
```

5.5 Twitter / X Scraper

Twitter/X public profiles can be scraped via the **Nitter** open-source frontend (self-hosted or public instances) which returns plain HTML without JavaScript requirements.

```
# Use a public Nitter instance:
https://nitter.net/{username}
https://nitter.privacydev.net/{username}
```

```
# Alternatively scrape twitter.com with Playwright
# Data extracted:
# display_name, @handle, bio, location, website,
# join_date, tweets_count, followers, following, pinned tweet text
```

5.6 GitHub Scraper

GitHub provides a fully free, unauthenticated REST API (v3) at `api.github.com` with a rate limit of 60 requests/hour. No API key is required.

```
# Endpoints (all free, no auth):
GET https://api.github.com/users/{username}
GET https://api.github.com/users/{username}/repos
GET https://api.github.com/users/{username}/orgs

# Data returned:
# name, blog, company, location, email (if public),
# bio, public_repos, followers, following,
# created_at, avatar_url, html_url
```

5.7 Email Discovery

Emails are discovered via three parallel strategies:

- **Hunter.io Free API:** `GET https://api.hunter.io/v2/email-finder?domain={domain}&first_name={first}&last_name={last}&api_key={key}` — 25 free requests/month
- **Google Dork:** search `"firstname.lastname" AND "@domain.com"` — parse email patterns from results
- **Pattern guessing + validation:** generate common formats (first.last@domain, flast@domain, etc.) and verify via SMTP or free MX validation services

5.8 Phone Number Discovery

- Google dork: `"target name" "phone" OR "mobile" OR "contact"`
- WHOIS lookup for company domain via `python-whois` — registrant phone sometimes exposed
- Parse any discovered web pages for phone number patterns using Python `phonenumbers` library

```
import phonenumbers, re

def extract_phones(text):
    pattern = r'[\+]?[1-9][0-9 .\-\(\)]{8,15}'
    candidates = re.findall(pattern, text)
    valid = []
    for c in candidates:
        try:
            p = phonenumbers.parse(c, None)
            if phonenumbers.is_valid_number(p):
                valid.append(
                    phonenumbers.format_number(p, phonenumbers.PhoneNumberFormat.IN
TERNATIONAL)
```

```
                )
        except:
            pass
    return list(set(valid))
```

6. Backend API Specification

6.1 Endpoints

Endpoint	Description	Response
POST /api/search	Submit a new search job	Returns job_id
GET /api/results/{job_id}	Poll or retrieve job results	Returns status + data
GET /api/jobs	List all past jobs for session	Array of jobs
DELETE /api/jobs/{job_id}	Delete a job and its data	204 No Content
GET /api/health	Service health check	200 OK
WebSocket /ws/{job_id}	Real-time progress stream	Progress events

6.2 POST /api/search — Request Body

```
{
  "inputs": {
    "full_name":       "John Smith",          // optional
    "email":           "john@example.com",    // optional
    "linkedin":        "johnsmith",           // optional
    "facebook":        "john.smith",          // optional
    "instagram":       "johnsmith_",          // optional
    "company_name":    "Acme Corp",           // optional
    "company_website": "acme.com",            // optional
    "personal_email":  "john@gmail.com",      // optional
    "city":            "New York",            // optional
    "country":         "USA",                 // optional
    "github":          "jsmith",             // optional
    "twitter":         "johnsmith"           // optional
  },
  "retrieve": [
    "personal_email", "work_email", "phone",
    "linkedin", "github", "twitter", "facebook",
    "instagram", "personal_website", "company"
  ]
}
// VALIDATION RULE: at least ONE field in 'inputs' must be non-null/non-empty
```

6.3 GET /api/results/{job_id} — Response

```
{
  "job_id":       "abc-123",
  "status":       "completed",   // pending | running | completed | failed
  "progress":     100,           // 0-100
  "created_at":   "2026-07-27T10:00:00Z",
  "completed_at": "2026-07-27T10:01:12Z",
  "results": {
    "personal_email":   ["john@gmail.com"],
    "work_email":       ["john.smith@acme.com"],
    "phone":            ["+1-212-555-0100"],
    "linkedin":         { "url": "...", "name": "...", "headline": "...", "locatio
n": "..." },
    "github":           { "url": "...", "repos": 42, "bio": "..." },
    "twitter":          { "url": "...", "handle": "@johnsmith", "bio": "..." },
    "facebook":         { "url": "...", "name": "...", "location": "..." },
    "instagram":        { "url": "...", "followers": 1200, "bio": "..." },
    "personal_website": ["https://johnsmith.dev"],
    "company":          { "name": "Acme Corp", "website": "acme.com", "role": "Engi
neer" }
  },
  "sources": [
    { "url": "https://linkedin.com/in/johnsmith", "scraped_at": "...", "confidenc
e": 0.95 }
  ]
}
```

7. Database Schema (PostgreSQL)

```
-- Jobs table
CREATE TABLE jobs (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id    VARCHAR(64),
  status        VARCHAR(20) DEFAULT 'pending',
  inputs        JSONB NOT NULL,
  retrieve      TEXT[] NOT NULL,
  progress      INT DEFAULT 0,
  created_at    TIMESTAMPTZ DEFAULT NOW(),
  completed_at  TIMESTAMPTZ,
  error_msg     TEXT
);

-- Results table
CREATE TABLE results (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id        UUID REFERENCES jobs(id) ON DELETE CASCADE,
  category      VARCHAR(50) NOT NULL,   -- e.g. 'linkedin', 'work_email'
  data          JSONB NOT NULL,
  source_url    TEXT,
  confidence    FLOAT DEFAULT 1.0,
  scraped_at    TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_results_job_id ON results(job_id);
CREATE INDEX idx_jobs_session   ON jobs(session_id);
```

8. Frontend Design (Next.js + Tailwind)

8.1 Page Structure

Route	Page Name	Purpose
/	Landing / Home	Hero, feature list, CTA to search
/search	Search Form	All 12 input fields + output checkboxes
/results/[jobId]	Results Page	Real-time results display by category
/history	Search History	Past searches with status

8.2 Search Form Component

The search form renders 12 optional input fields in a 2-column grid layout. The Submit button is disabled until at least one field has a non-empty value. Client-side validation is handled by React Hook Form + Zod.

```
// Zod schema (abbreviated)
const schema = z.object({
  full_name:       z.string().optional(),
  email:           z.string().email().optional().or(z.literal('')),
  linkedin:        z.string().url().optional().or(z.literal('')),
  // ... all other fields ...
  retrieve:        z.array(z.string()).min(1, 'Select at least one output'),
}).refine(
  (data) => Object.values(data).some(v => v && v !== ''),
  { message: 'At least one input field is required' }
);
```

8.3 Output Checkboxes

Below the input fields, 10 styled checkboxes (shadcn/ui Checkbox) let the user pick which data categories to retrieve. All are checked by default. Unchecking categories not needed speeds up the search.

8.4 Real-Time Progress

After submission the frontend connects to `ws://{server}/ws/{job_id}`. The server emits progress events as each scraper completes. A progress bar and per-category status indicators update live. When a scraper finishes its results render immediately, without waiting for all others.

8.5 Results Display

Results are grouped into collapsible cards per category (LinkedIn, Email, GitHub, etc.). Each card shows scraped data fields, source URL, scrape timestamp, and a confidence badge (High / Medium / Low). Users can export results as JSON or CSV.

9. Celery Task Design (Background Workers)

9.1 Task Chain Architecture

When a job is submitted, the orchestrator task fans out to individual scraper sub-tasks in parallel using Celery's `chord` primitive. All results are collected and merged in a final callback.

```
# tasks/orchestrator.py
from celery import chord
from .scrapers import *

@celery_app.task
def run_search(job_id: str, inputs: dict, retrieve: list):
    task_map = {
        'linkedin':        scrape_linkedin.s(job_id, inputs),
        'github':          scrape_github.s(job_id, inputs),
        'twitter':         scrape_twitter.s(job_id, inputs),
        'facebook':        scrape_facebook.s(job_id, inputs),
        'instagram':       scrape_instagram.s(job_id, inputs),
        'personal_email':  scrape_emails.s(job_id, inputs),
        'work_email':      scrape_emails.s(job_id, inputs),
        'phone':           scrape_phone.s(job_id, inputs),
        'personal_website':scrape_website.s(job_id, inputs),
        'company':         scrape_company.s(job_id, inputs),
    }
    tasks = [task_map[r] for r in retrieve if r in task_map]
    chord(tasks)(merge_results.s(job_id=job_id))
```

9.2 Error Handling

- Each scraper task has `max_retries=3` with exponential backoff
- If a scraper fails after retries, it stores an empty result with `error=True` — it does not crash the entire job
- The `merge_results` callback runs regardless of individual task failures
- Jobs stuck in 'running' for more than 5 minutes are auto-failed by a periodic Celery Beat cleanup task

10. Anti-Detection & Rate Limiting Strategy

10.1 Request Rotation

- **User-Agent rotation:** Use `fake-useragent` library to rotate across hundreds of real browser UA strings
- **Referer spoofing:** Set Referer header to a plausible origin (e.g. google.com) per request
- **Accept-Language randomisation:** Rotate locale headers
- **Request timing:** Add random delays of 2–8 seconds between requests per domain

10.2 Proxy Support (Free Options)

- **Free proxy lists:** scrapeninja.net, proxyscrape.com — auto-fetch and validate proxies at startup
- **Tor network:** route requests through Tor SOCKS5 proxy (127.0.0.1:9050) for sensitive targets
- **ScraperAPI free plan:** 1,000 free API calls/month at scraperapi.com — handles rotation automatically

10.3 Playwright Stealth

```
pip install playwright-stealth

from playwright.async_api import async_playwright
from playwright_stealth import stealth_async

async with async_playwright() as p:
    browser = await p.chromium.launch(headless=True)
    page    = await browser.new_page()
    await stealth_async(page)  # patches navigator.webdriver, plugins, etc.
    await page.goto(target_url)
```

10.4 Respectful Crawling

> *Always check robots.txt before scraping. Limit request rate to maximum 1 request per 3 seconds per domain. Cache responses in Redis for 24 hours to avoid redundant requests for the same target.*

11. Project Directory Structure

```
public-intelligence/
├── frontend/                      # Next.js application
│   ├── app/
│   │   ├── page.tsx               # Landing page
│   │   ├── search/page.tsx        # Search form
│   │   └── results/[jobId]/page.tsx  # Results view
│   ├── components/
│   │   ├── SearchForm.tsx
│   │   ├── OutputCheckboxes.tsx
│   │   ├── ResultCard.tsx
│   │   └── ProgressBar.tsx
│   ├── lib/api.ts                 # Axios API client
│   └── package.json
│
├── backend/                       # FastAPI application
│   ├── main.py                    # FastAPI entry point
│   ├── routers/
│   │   ├── search.py              # POST /api/search
│   │   └── results.py             # GET /api/results/{id}
│   ├── models/
│   │   ├── job.py                 # SQLAlchemy Job model
│   │   └── result.py              # SQLAlchemy Result model
│   ├── schemas/
│   │   ├── search_request.py      # Pydantic input schema
│   │   └── result_response.py     # Pydantic output schema
│   ├── tasks/
│   │   ├── celery_app.py          # Celery configuration
│   │   ├── orchestrator.py        # Job fan-out logic
│   │   └── scrapers/
│   │       ├── google.py
│   │       ├── linkedin.py
│   │       ├── facebook.py
│   │       ├── instagram.py
│   │       ├── twitter.py
│   │       ├── github.py
│   │       ├── email_finder.py
│   │       └── phone.py
│   ├── db.py                      # SQLAlchemy session setup
│   ├── config.py                  # Settings via pydantic-settings
│   └── requirements.txt
│
├── docker-compose.yml             # Local dev orchestration
├── .env.example                   # Environment variable template
└── README.md
```

12. Docker Compose — Local Development

```
# docker-compose.yml
version: '3.9'
services:

  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: osint_db
      POSTGRES_USER: osint_user
      POSTGRES_PASSWORD: osint_pass
    ports: ['5432:5432']
    volumes: [postgres_data:/var/lib/postgresql/data]

  redis:
    image: redis:7-alpine
    ports: ['6379:6379']

  backend:
    build: ./backend
    command: uvicorn main:app --host 0.0.0.0 --port 8000 --reload
    ports: ['8000:8000']
    env_file: .env
    depends_on: [postgres, redis]

  worker:
    build: ./backend
    command: celery -A tasks.celery_app worker --loglevel=info --concurrency=4
    env_file: .env
    depends_on: [postgres, redis]

  frontend:
    build: ./frontend
    command: npm run dev
    ports: ['3000:3000']
    environment:
      NEXT_PUBLIC_API_URL: http://localhost:8000

volumes:
  postgres_data:
```

13. Environment Variables (.env)

```
# .env.example

# Database
DATABASE_URL=postgresql://osint_user:osint_pass@postgres:5432/osint_db

# Redis / Celery
REDIS_URL=redis://redis:6379/0
CELERY_BROKER_URL=redis://redis:6379/0
CELERY_RESULT_BACKEND=redis://redis:6379/1

# Free APIs
HUNTER_IO_API_KEY=your_free_hunter_key           # hunter.io free plan
GOOGLE_CSE_API_KEY=your_google_cse_key           # free 10k/day
GOOGLE_CSE_CX=your_custom_search_engine_id

# Optional proxy
HTTP_PROXY=socks5://127.0.0.1:9050               # Tor (optional)

# Security
SECRET_KEY=your_random_secret_key_here
ALLOWED_ORIGINS=http://localhost:3000
```

14. CI/CD Pipeline (GitHub Actions — Free)

```
# .github/workflows/deploy.yml
name: Build & Deploy
on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12' }
      - run: pip install -r backend/requirements.txt
      - run: pytest backend/tests/

  deploy-backend:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: railwayapp/railway-action@v1
        with:
          service: backend
          token: ${{ secrets.RAILWAY_TOKEN }}

  deploy-frontend:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: amondnet/vercel-action@v25
        with:
          vercel-token:      ${{ secrets.VERCEL_TOKEN }}
          vercel-org-id:     ${{ secrets.ORG_ID }}
          vercel-project-id: ${{ secrets.PROJECT_ID }}
```

15. Step-by-Step Build Guide for Engineers

Phase 1 — Project Setup (Day 1)

1. Create GitHub repository
2. Clone repo locally
3. Create `docker-compose.yml` as defined in Section 12
4. Create `.env` from `.env.example`
5. Run: `docker-compose up -d`
6. Verify PostgreSQL and Redis are running

Phase 2 — Backend API (Days 2–3)

1. Initialise FastAPI project in `/backend`
2. Install all packages from `requirements.txt`
3. Create SQLAlchemy models (Job, Result) and run Alembic migrations
4. Implement `POST /api/search` endpoint with Pydantic validation
5. Implement `GET /api/results/{job_id}` endpoint
6. Add WebSocket endpoint for progress streaming
7. Write pytest unit tests for validation logic

Phase 3 — Celery Workers (Days 4–6)

1. Configure Celery with Redis broker in `tasks/celery_app.py`
2. Implement orchestrator task with `chord` fan-out
3. Build Google/DuckDuckGo dork scraper
4. Build GitHub scraper using free GitHub API
5. Build Twitter/X scraper using Nitter
6. Build LinkedIn, Facebook, Instagram scrapers using Playwright + scrapy-playwright
7. Build email discovery (Hunter.io + pattern matching)
8. Build phone number extractor
9. Test each scraper independently with a known public profile

Phase 4 — Frontend (Days 7–9)

1. Initialise Next.js 14 project with Tailwind and shadcn/ui
2. Build SearchForm component with all 12 input fields

3. Add output checkbox group (10 categories)
4. Implement Zod + React Hook Form validation (min 1 field rule)
5. Connect to backend `POST /api/search`
6. Build Results page with collapsible category cards
7. Add WebSocket listener for live progress updates
8. Add JSON/CSV export functionality
9. Implement responsive mobile layout

Phase 5 — Deployment (Day 10)

1. Push code to GitHub
2. Create Railway.app project — connect GitHub repo — deploy backend + Postgres + Redis
3. Create Vercel project — connect GitHub repo — deploy frontend
4. Set all environment variables in Railway and Vercel dashboards
5. Run end-to-end test with a real public profile
6. Set up GitHub Actions CI/CD workflow

16. Legal & Ethical Considerations

> ***MANDATORY READING:*** *The engineer and operator of this system are solely responsible for ensuring legal compliance in their jurisdiction.*

16.1 What Is Permitted

- Accessing only publicly available, unauthenticated information
- Data that the individuals themselves have voluntarily made public on social media platforms
- Business use cases such as lead generation, journalistic research, recruitment, and background checks where permitted by local law

16.2 What Is Prohibited

- Bypassing any authentication or login wall
- Scraping private profiles or data not intended for public access
- Violating platform Terms of Service — review each platform's ToS before deployment
- Storing or selling personal data in jurisdictions that prohibit this (GDPR, CCPA, PIPEDA, etc.)
- Stalking, harassment, doxxing, or any activity that harms the subject
- Scraping EU residents' data without a valid legal basis under GDPR Article 6

16.3 Recommended Safeguards

- Add a mandatory "I agree to use this tool lawfully" checkbox before every search
- Log all searches with timestamps for auditing
- Implement rate limiting per user account (e.g. max 50 searches/day)
- Auto-delete stored results after 30 days
- Add a public Privacy Policy and Terms of Service page
- Allow subjects to request deletion of their cached data (GDPR Right to Erasure)

17. Total Cost Summary

Service	Purpose	Cost
Railway.app	Backend hosting + Redis + Postgres	**$0 / month**
Vercel	Frontend hosting	**$0 / month**
Neon.tech	Serverless PostgreSQL	**$0 / month**
Upstash Redis	Serverless Redis	**$0 / month**
GitHub	Source control + CI/CD	**$0 / month**
GitHub API	Public data scraping	**$0 / unlimited**
Hunter.io	Email discovery	**$0 / 25 req per month**
Google CSE	Custom Search API	**$0 / 10,000 queries/day**
DuckDuckGo	Search scraping	**$0 / unlimited**
Nitter	Twitter/X scraping	**$0 / self-hostable**
All npm packages	Frontend dependencies	**$0 / open-source**
All Python packages	Backend dependencies	**$0 / open-source**
TOTAL	**Full stack deployment**	**$0 / month**

18. Glossary

Term	Definition
OSINT	Open Source Intelligence — gathering information from publicly available sources
Scraper	A program that automatically extracts data from web pages
Dork	A crafted search query using advanced operators to find specific information
Celery	A distributed task queue library for Python used to run background jobs
Redis	An in-memory data store used as message broker and cache
FastAPI	A modern, fast Python web framework for building REST APIs
Playwright	A browser automation library that controls headless Chrome/Firefox/Safari
WHOIS	A protocol that returns registration info for domain names
chord (Celery)	A group of tasks that run in parallel with a callback on completion
Nitter	An open-source alternative Twitter frontend that serves plain HTML
GDPR	General Data Protection Regulation — EU data privacy law
CCPA	California Consumer Privacy Act — US state data privacy law

End of Document | Public Intelligence SaaS Blueprint v1.0 | July 2026

Built for engineers and AI agents. Use responsibly and in compliance with applicable law.